Milestone 1: Recuperação lexical BM25 sobre o PubMedQA

Universidade de Évora · Mestrado em Engenharia Informática · 2026/2027
Unidade curricular: Recuperação de Informação em Bases de Texto · Docente: Paulo Quaresma
Autor: Rafael da Conceição Serrão, n.º 76135 (trabalho individual)
Repositório: https://github.com/rafaelserrao/mestrado_recuperacao_bases_texto

1. Objetivo

O objetivo desta primeira fase era ter uma baseline lexical a funcionar para o projeto MedIR-RAG, que vai servir de comparação na Milestone 2 (recuperação densa e re-ranking). Para isso, o PubMedQA foi indexado com BM25, foram testadas três formas de pré-processar o texto e vários valores de k1 e b, e os resultados foram avaliados com o `pytrec_eval` (NDCG@5, NDCG@10, MRR@10, MAP e Recall@100).

2. Dados e formulação da tarefa

Das duas coleções propostas foi escolhido o PubMedQA (Jin et al., 2019). Como o dataset foi pensado para question answering e não para recuperação, foi preciso adaptá-lo. Foi usado o subconjunto anotado PQA-L, com 1 000 perguntas, da seguinte forma:

- Query: o texto da pergunta (`QUESTION`).
- Documento: o abstract do artigo (`CONTEXTS` + `LONG_ANSWER`), identificado pelo PMID.
- Qrels: cada pergunta tem um só documento relevante, que é o abstract de onde a pergunta saiu (relevância binária).

Foram montados dois corpora. O `pqal` tem apenas os 1 000 abstracts do PQA-L e, por isso, é bastante fácil. O `pqal+pqau` acrescenta 61 249 abstracts do PQA-U (a parte não anotada), que funcionam como distratores, e fica com 62 249 documentos. Este segundo cenário está mais perto de uma pesquisa real, em que o abstract certo aparece misturado com muitos outros parecidos.

3. Metodologia

Foram comparadas três pipelines de pré-processamento, cada uma a acrescentar um passo à anterior. As mesmas regras foram aplicadas aos documentos e às perguntas.

1. `lower`: passar o texto para minúsculas e separá-lo em tokens com uma expressão regular (só tokens alfanuméricos com pelo menos 2 caracteres).
2. `stop`: igual ao anterior, sem as stopwords em inglês.
3. `stop_stem`: igual ao `stop`, com stemming Snowball (PyStemmer).

No corpus completo, tirar as stopwords baixa o tamanho médio dos documentos de 237 para 173 tokens, e o stemming reduz o vocabulário de cerca de 104 mil para 80 mil termos.

Para o BM25 foi usada a biblioteca `bm25s` (Lù, 2024) na variante Lucene, que é a mesma fórmula do Pyserini. A escolha teve a ver com não precisar de Java, ao contrário do Pyserini e do PyTerrier, e com a velocidade: o `bm25s` calcula os scores logo na indexação e guarda-os numa matriz esparsa. Para cada pergunta foram guardados os 100 primeiros documentos, em ficheiros no formato TREC.

Os parâmetros foram testados numa grelha com $k1 \in \{0{,}9; 1{,}2; 1{,}5\}$ e $b \in \{0{,}4; 0{,}75\}$, o que dá 36 configurações no total (2 corpora × 3 pipelines × 6 pares). No corpus completo, a indexação demorou cerca de 3 s e a pesquisa das 1 000 perguntas entre 0,4 e 0,6 s, num portátil e só com CPU.

A avaliação foi feita com o `pytrec_eval`, com as medidas `ndcg_cut_5`, `ndcg_cut_10`, `map` e `recall_100`. O MRR@10 deu um pouco mais de trabalho, porque o trec_eval não tem uma versão do `recip_rank` com corte. A solução foi cortar cada run aos 10 primeiros resultados e calcular aí o `recip_rank`. Perguntas sem nenhum resultado contam como 0. Para perceber se as diferenças entre configurações eram reais ou só ruído, foi ainda usado o teste de Wilcoxon emparelhado por pergunta.

4. Resultados

No corpus completo, a melhor configuração foi `stop_stem` com k1 = 0,9 e b = 0,75, com NDCG@10 = 0,911 e MRR@10 = 0,891. A Tabela 1 mostra a melhor combinação de k1 e b para cada pipeline, escolhida pelo NDCG@10. Os valores das 36 configurações estão em `results/results.md`.

Tabela 1. Melhor configuração por corpus e pipeline.

Corpus	Pré-proc.	k1	b	NDCG@5	NDCG@10	MRR@10	MAP	R@100
pqal	lower	0,9	0,75	0,981	0,981	0,979	0,979	0,995
pqal	stop	0,9	0,75	0,981	0,982	0,979	0,980	0,997
pqal	stop_stem	1,2	0,75	**0,986**	**0,987**	**0,985**	**0,985**	0,997
pqal+pqau	lower	0,9	0,75	0,899	0,905	0,888	0,889	0,985
pqal+pqau	stop	0,9	0,75	0,896	0,903	0,884	0,885	0,985
pqal+pqau	stop_stem	0,9	0,75	**0,905**	**0,911**	**0,891**	**0,892**	**0,990**

Com os parâmetros por omissão (k1 = 1,2; b = 0,75) e `stop_stem`, o corpus completo fica com NDCG@10 = 0,906 e MRR@10 = 0,886.

Os testes de Wilcoxon (`results/significance.md`) mudaram um pouco a leitura da tabela. No corpus completo, a única diferença significativa foi baixar o k1 de 1,2 para 0,9 em `stop_stem`: o NDCG@10 sobe 0,004, com p = 0,002. O stemming parece ajudar face ao `lower` (+0,006 de NDCG@10), mas com p = 0,42 não se pode dizer que a diferença seja real. No `pqal` o stemming já é significativo (p = 0,035), embora a diferença também seja pequena, de 0,006.

5. Discussão

Como cada pergunta tem só um documento relevante, a average precision fica igual a 1 a dividir pela posição desse documento, ou seja, é o reciprocal rank sem corte. Por isso o MAP e o MRR@10 dão valores quase iguais e só se afastam quando o relevante aparece entre a posição 11 e a 100. Esta coincidência ajudou a confirmar que a avaliação estava bem feita. Nesta formulação, as métricas que trazem mais informação são o NDCG@k e o Recall@100.

Os distratores fazem bastante diferença. Com os 61 mil abstracts extra, o MRR@10 cai cerca de 9 pontos (de 0,985 para 0,891). No `pqal` quase todas as configurações acertam e é difícil distingui-las, por isso nas próximas milestones a referência vai ser o `pqal+pqau`.

Quanto ao pré-processamento, o stemming junta variantes da mesma palavra (por exemplo "remodelling" e "remodel") e teve a melhor média, mas no corpus completo a diferença não é significativa. Tirar as stopwords praticamente não muda os resultados, o que faz sentido, porque o IDF do BM25 já dá pouco peso a palavras muito frequentes. Nos parâmetros, o k1 mais baixo (0,9) foi o melhor no corpus completo. Com k1 baixo, a frequência de cada termo satura mais cedo e passa a contar mais quantos termos diferentes da pergunta aparecem no abstract. O b quase não teve efeito.

Convém notar que estes valores são otimistas. Muitas perguntas do PubMedQA foram feitas a partir do título do artigo e repetem várias palavras do abstract, o que facilita a vida a um método lexical como o BM25.

Na análise de erros (melhor configuração, corpus completo), o relevante aparece em 1.º lugar em 84,3 % das perguntas, entre a 2.ª e a 10.ª posição em 12,7 %, entre a 11.ª e a 100.ª em 2,0 % e fica fora do top-100 em 1,0 %. Olhando para algumas das falhas, aparecem sobretudo dois casos. Um são temas muito estudados no PubMed, em que há vários abstracts com os mesmos termos ("aromatase inhibitors … breast cancer", "type 2 diabetes"). O outro são perguntas retóricas ou mais coloquiais, com poucas palavras em comum com o abstract, como "Lloyd-Davies position with Trendelenburg--a disaster waiting to happen?". A ideia é que a recuperação densa e o re-ranking da Milestone 2 ajudem precisamente nestes casos.

6. Reprodução e uso de IA

O código, os testes e as instruções para correr tudo estão no repositório indicado no cabeçalho (ver o README). O `scripts/download_data.py` descarrega os dados e o `scripts/run_experiments.py` corre a grelha toda e gera a pasta `results/`. Os runs ficam em `runs/`, no formato TREC.

Ao longo do trabalho foi usado o Claude Code (Anthropic) como apoio na escrita do código e na revisão do texto deste relatório. As decisões sobre as experiências, a execução e a análise dos resultados foram verificadas por mim.

Referências

Jin, Q. et al. PubMedQA: A Dataset for Biomedical Research Question Answering. EMNLP 2019. · Lù, X. H. BM25S: Orders of magnitude faster lexical search via eager sparse scoring. 2024. · Van Gysel, C. & de Rijke, M. Pytrec_eval: An Extremely Fast Python Interface to trec_eval. SIGIR 2018. · Robertson, S. & Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR, 2009.